

LAB-8

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

- Compiled maven project should be pushed to github profile.
- Jenkins CI Pipeline (build Maven project)
- Artifact generation (.jar/.war)
- Ansible deployment from Ubuntu

PART 1: Jenkins Setup (CI Pipeline)

1. Install Required Plugins in Jenkins

Go to:

Manage Jenkins → Plugins → Available Plugins

Install:

- Git Plugin
- Maven Integration Plugin
- Pipeline Plugin --. pipeline stage view
- SSH Agent Plugin

Restart Jenkins after install.

2. Install git in ubuntu terminal

```
sudo apt update
sudo apt install git -y
which git
/usr/bin/git
```

a. Configure Git in Jenkins

Go to:

Manage Jenkins → Global Tool Configuration

- Scroll to **Git**
- Click **Add Git**
- Name: Default
- Path to Git executable: `/usr/bin/git` (or your path)

```
sudo systemctl restart jenkins
```

3. Configure Tools in Jenkins

Go to:

Manage Jenkins → Global Tool Configuration

Add Maven

- Name: Maven
- Select: Install automatically

Add JDK

- Name: JDK
- Install automatically

3. Create Jenkins Pipeline Job

Go to:

New Item → Pipeline → Name: Maven-CI-CD → OK

4. Configure GitHub Repository

Inside job:

- Go to **Pipeline → Definition**
- Select: Pipeline script
- below is the jenkins file

```
pipeline {
  agent any

  tools {
    maven 'Maven'
    jdk 'JDK'
  }

  stages {

    stage('Clean') {
      steps {
        cleanWs()
      }
    }

    stage("shell script"){
      steps{
        sh 'whoami'
        sh 'ls -l /home/lab/deploy/'
      }
    }

    stage('clone repository and Checkout') {
      steps {
        git branch: 'main', url:
```

```

https://github.com/bhargavi-
    }
  }

  stage('Build') {
    steps {
      sh 'mvn clean install'
    }
  }

  stage('Copy Artifact') {
    steps {
      sh 'cp target//lab2-1.0-SNAPSHOT.jar
/home/lab/deploy/'
    }
  }

  stage('Run Ansible Playbook') {
    steps {
      sh 'ansible-playbook /home/lab/deploy/deploy.yml -
i /home/lab/deploy/hosts'
    }
  }
}

```

6. Build Job

Click:

Build Now

Check Console Output → Ensure:

- Maven build success
- .jar created in target/

PART 2: Ubuntu (Ansible Setup)

1. Install Ansible

On Ubuntu:

sudo apt update

sudo apt install ansible -y

ansible --version

check present working directory:

pwd

o/p: /home/lab

2. Create Project Structure

```
mkdir -p ~/deploy
cd ~/deploy
```

3. Create Inventory File (inside deploy directory)

```
nano hosts
[app]
localhost ansible_connection=local
```

4. Create Ansible Playbook (inside deploy directory)

```
nano deploy.yml
- name: Deploy Maven Artifact Locally
  hosts: localhost
  connection: local

vars:
  deploy_dir: /home/lab/deploy/myapp # Where the app will run
  source_dir: /home/lab/deploy/      # Where Jenkins copied the JAR

tasks:

  - name: Create deployment directory
    file:
      path: "{{ deploy_dir }}"
      state: directory

  - name: Stop existing app if running
    shell: pkill -f 'java -jar'
    register: pkill_result
    ignore_errors: yes
    changed_when: pkill_result.rc == 0

  - name: Copy JAR file to deployment directory
    copy:
      src: "/home/lab/deploy/lab2-1.0--SNAPSHOT.jar"
      dest: "{{ deploy_dir }}/lab2-1.0-SNAPSHOT.jar"
      remote_src: yes

  - name: Run new JAR
    shell: "nohup java -jar {{ deploy_dir }}/*.jar > {{ deploy_dir }}/app.log 2>&1 &"
```

```
lab@HP-PRODESK: ~/deploy
GNU nano 7.2 deploy.yml
name: Deploy Maven Artifact Locally
hosts: localhost
connection: local

vars:
  deploy_dir: /home/lab/deploy/myapp # Where the app will run
  source_dir: /home/lab/deploy/      # Where Jenkins copied the JAR

tasks:
  - name: Create deployment directory
    file:
      path: "{{ deploy_dir }}"
      state: directory

  - name: Stop existing app if running
    shell: pkill -f 'java -jar'
    register: pkill_result
    ignore_errors: yes
    changed_when: pkill_result.rc == 0

  - name: Copy JAR file to deployment directory
    copy:
      src: "/home/lab/deploy/lab2-1.0-SNAPSHOT.jar"
      dest: "{{ deploy_dir }}/lab2-1.0-SNAPSHOT.jar"
      remote_src: yes

[ Read 30 lines ]
```

5. Give Permissions

`chmod +x deploy.yml`

a. NOW go to jenkins pipeline script add the blocks “copy artifact” and “run ansible playbook” in pipeline script

1. Jenkins builds project (mvn clean package)
2. JAR gets created
3. Jenkins calls Ansible
4. Ansible deploys JAR

b. In ubuntu Terminal

`sudo chown -R jenkins:jenkins /home/lab/deploy`

`sudo chmod -R 777 /home/lab/deploy`

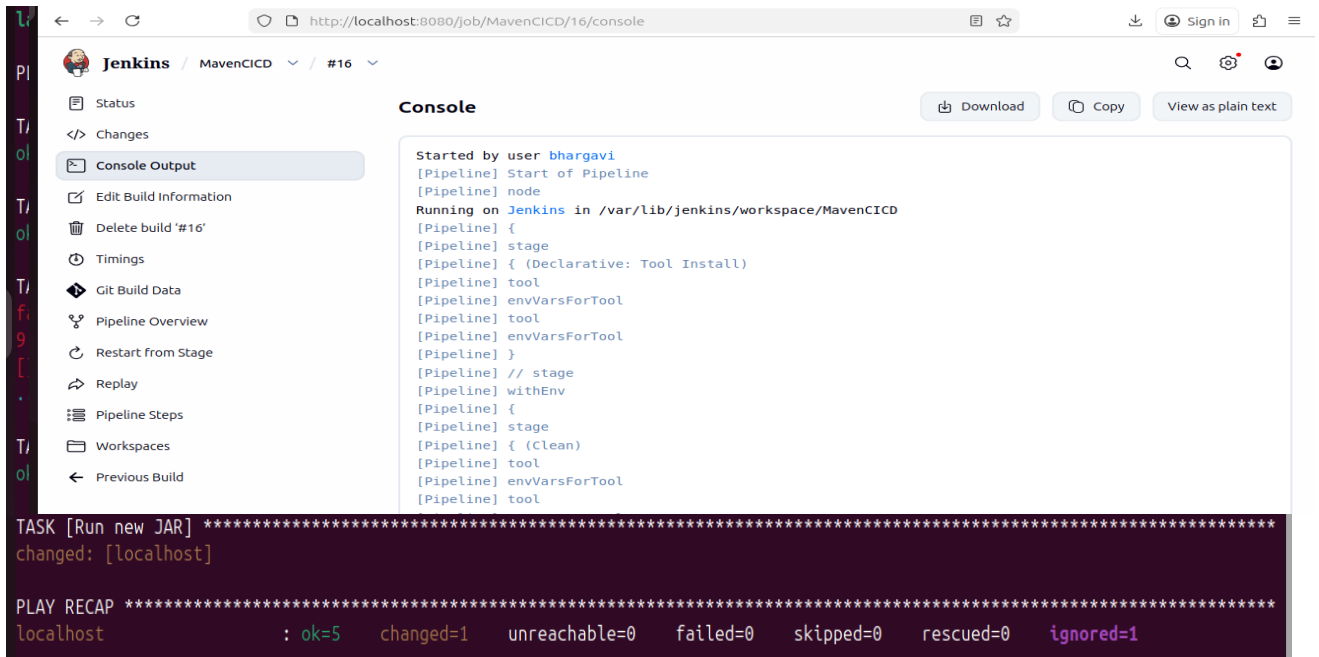
`sudo usermod -aG lab jenkins`

`sudo systemctl restart Jenkins`

c. go to Jenkins job and give “BUILD NOW”

6. Test Playbook Manually

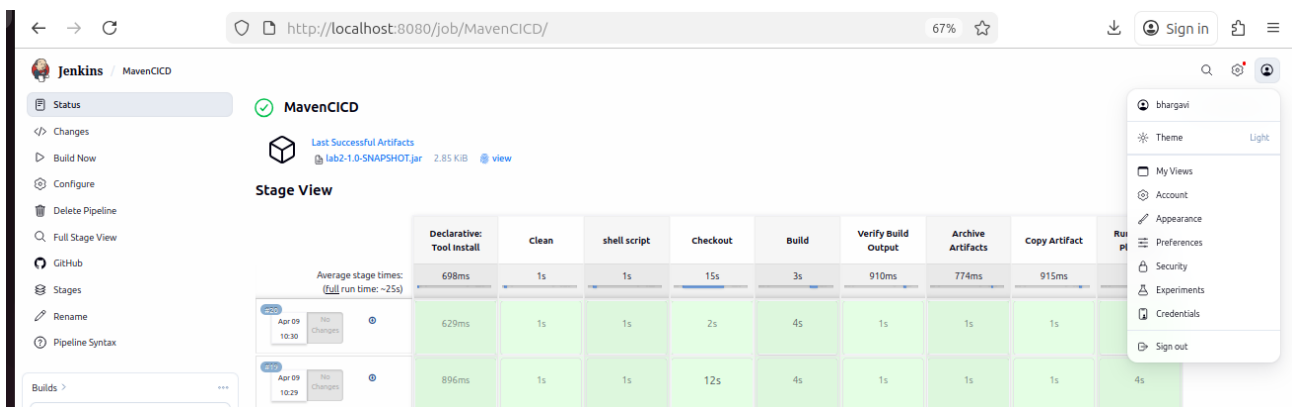
`ansible-playbook -i hosts deploy.yml`



go back to jenkins job Maven-CI-CD and click on build now
 artifact output

```
lab@HP-PRODESK:~/deploy$ ls
deploy.yml  hosts  lab2-1.0-SNAPSHOT.jar
```

pipeline stage view output

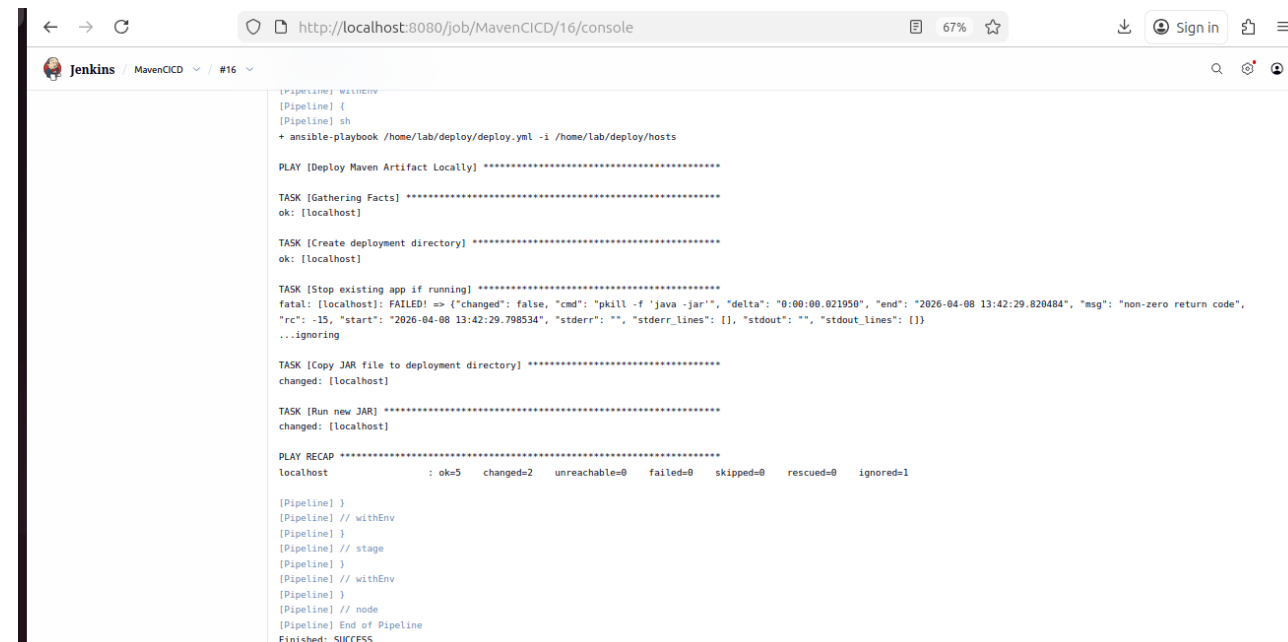


pipeline console output



The screenshot shows the Jenkins console output for a Maven build. The URL in the browser is `http://localhost:8080/job/MavenCICD/16/console`. The output shows the following steps:

```
[[1:34mINFO[m]
[[1:34mINFO[m] .....
[[1:34mINFO[m] T E S T S
[[1:34mINFO[m] .....
[[1:34mINFO[m] Running com.yit.demo.[1mAppTest[m
Running test: 2 + 3 = 5
[[1:34mINFO[m] [1:32mTests run: 0;1:32m[m, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in com.yit.demo.[1mAppTest[m
[[1:34mINFO[m]
[[1:34mINFO[m] Results:
[[1:34mINFO[m]
[[1:34mINFO[m] [1:32mTests run: 1, Failures: 0, Errors: 0, Skipped: 0[m
[[1:34mINFO[m]
[[1:34mINFO[m]
[[1:34mINFO[m] [1m--- [0:32mmaven-jar-plugin:3.4.2:jar[m [1mdefault-jar][m @ [36mlab2[0:1m ---[m
[[1:34mINFO[m] Building jar: /var/lib/jenkins/workspace/MavenCICD/target/lab2-1.0-SNAPSHOT.jar
[[1:34mINFO[m]
[[1:34mINFO[m] [1m--- [0:32mmaven-install-plugin:3.1.2:install[m [1mdefault-install][m @ [36mlab2[0:1m ---[m
[[1:34mINFO[m] Installing /var/lib/jenkins/workspace/MavenCICD/pom.xml to /var/lib/jenkins/.m2/repository/com/yit/lab2/1.0-SNAPSHOT/lab2-1.0-SNAPSHOT.pom
[[1:34mINFO[m] Installing /var/lib/jenkins/workspace/MavenCICD/target/lab2-1.0-SNAPSHOT.jar to /var/lib/jenkins/.m2/repository/com/yit/lab2/1.0-SNAPSHOT/lab2-1.0-SNAPSHOT.jar
[[1:34mINFO[m] [1m-----[m
[[1:34mINFO[m] [1:32mBUILD SUCCESS[m
[[1:34mINFO[m] [1m-----[m
[[1:34mINFO[m] Total time: 2.801 s
[[1:34mINFO[m] Finished at: 2026-04-08T13:42:24+05:30
[[1:34mINFO[m] [1m-----[m
[Pipeline] }
[Pipeline] // withEnv
```



The screenshot shows the Jenkins console output for an Ansible deployment. The URL in the browser is `http://localhost:8080/job/MavenCICD/16/console`. The output shows the following steps:

```
[Pipeline] WALHEDEV
[Pipeline] {
[Pipeline] sh
+ ansible-playbook /home/lab/deploy/deploy.yml -i /home/lab/deploy/hosts

PLAY [Deploy Maven Artifact Locally] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create deployment directory] *****
ok: [localhost]

TASK [Stop existing app if running] *****
fatal: [localhost]: FAILED! => {"changed": false, "cmd": "pkill -f 'java -jar'", "delta": "0:00:00.021950", "end": "2026-04-08 13:42:29.020484", "msg": "non-zero return code", "rc": -15, "start": "2026-04-08 13:42:29.798534", "stderr": "", "stderr_lines": [], "stdout": "", "stdout_lines": []}
...ignoring

TASK [Copy JAR file to deployment directory] *****
changed: [localhost]

TASK [Run new JAR] *****
changed: [localhost]

PLAY RECAP *****
localhost      : ok=5  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=1

[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

PART 3: Full Flow (What Happens)

1. Jenkins pulls code from GitHub
2. Maven builds project (`mvn clean install`)
3. `.jar` file created in `target/`
4. Jenkins copies `.jar` → Ubuntu (Ansible server)
5. Jenkins triggers Ansible
6. Ansible:
 - Copies file to `/home/lab/deploy/myapp`
 - Stops old app and Starts new app